

CS1CA Computer Systems Architecture

Data manipulation (3)

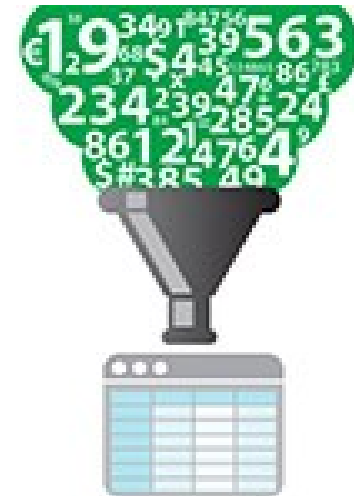
Week 4 lecture 1 of the autumn semester 2025

Lectured by

Dr. Hong Wei

h.wei@reading.ac.uk

Room 133, Polly Vacher



Outline of the topics

- Computer architecture: introduction
- Memory organisation
- Instruction encoding and decoding
 - Introduction to a simple machine
 - A simple machine language
- **Instruction execution cycle**

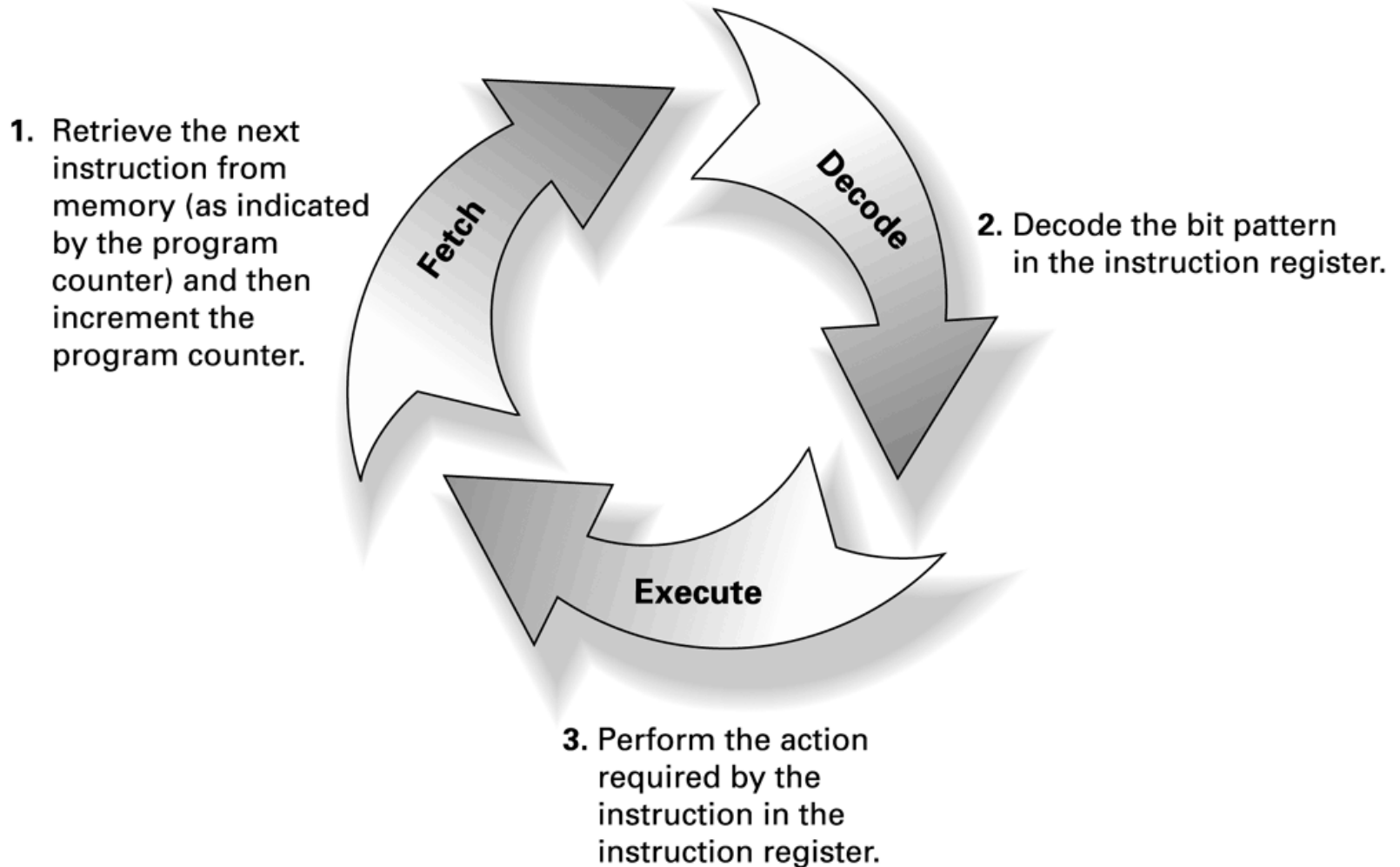
In this week

- The following contents are delivered.
 - Instruction execution cycles
- The learning outcomes are listed below.
 - To enhance the knowledge of instruction executions
 - To enhance the knowledge of data manipulations during instruction execution

Instruction execution

- Instruction execution is controlled by two special purpose registers (SPRs) in the CPU.
 - **Program Counter (PC)**: contain a memory address of the next instruction to be executed
 - **Instruction register (IR)**: contain the currently executed instruction
- Machine execution cycle
 - Instruction fetch
 - Instruction decoding
 - Execution

Execution cycle



PC and IR in instruction execution

- Both instructions and data are stored in the main memory during program execution.
- To execute an instruction, the content of **PC** indicates where to fetch the next instruction, and then the instruction is loaded into **IR**.
- Example instruction: **Load R1, (D7); // 0x11D7**
 - Access the memory address defined by **PC** to fetch the instruction, place it in **IR**. Update **PC**.
 - Decoding the code in **IR** found it is a “load” instruction.
 - Access the memory address defined by D7 to load its content from the memory
 - Place the loaded data into register R1.

Instruction execution – a simple task

- Add two values (operands) stored in memory cells and put the result back to another memory cell
 - Step 1: load the first operand to be added from memory and place it in a register
 - Step 2: load the second operand to be added from memory and place it in another register
 - Step 3: activate the addition circuitry with the registers used in Steps 1&2 as inputs and another register to hold the result
 - Step 4: store the result in memory
 - Step 5: stop

Operations involved in the adding

- We use the following instruction sequence to conduct the operations.

An assembly language expression to conduct the adding operation is list below:

Load R5, (6C)

Load R6, (6D)

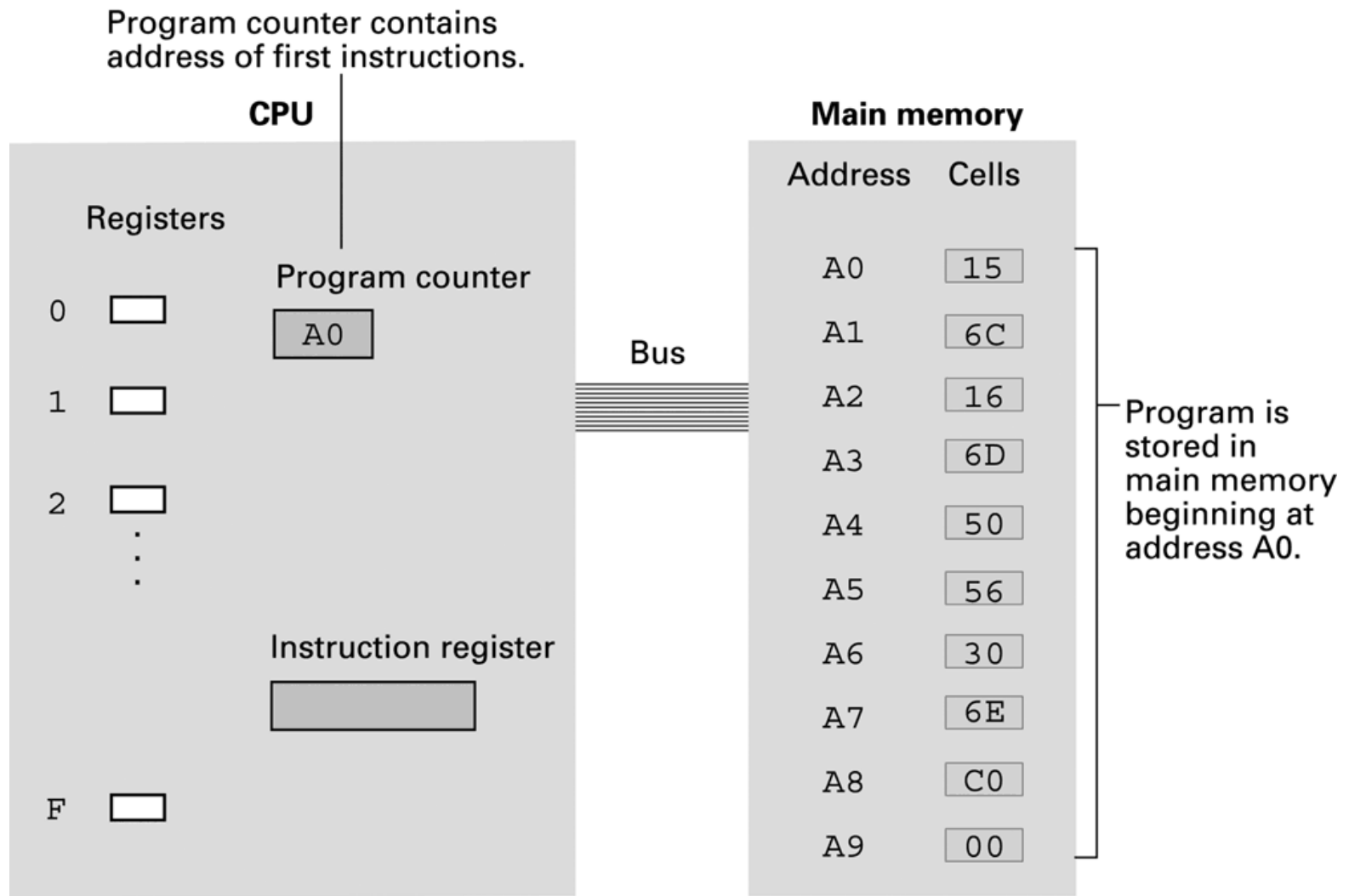
Add R0, R5, R6

Store (6E), R0

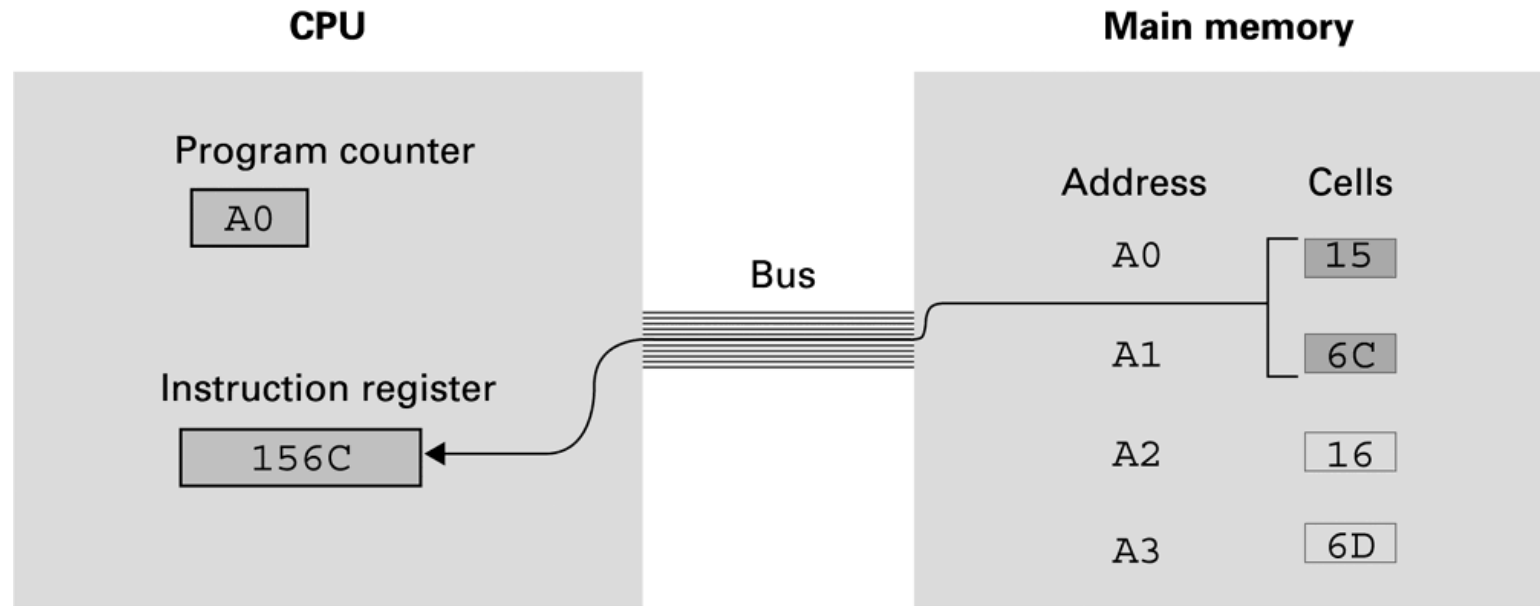
Halt

	Encoded instructions	Translation
Step 1	156C	Load register 5 with the bit pattern found in the memory cell at address 6C.
Step 2	166D	Load register 6 with the bit pattern found in the memory cell at address 6D.
Step 3	5056	Add the contents of register 5 and 6 as though they were two's complement representation and leave the result in register 0.
Step 4	306E	Store the contents of register 0 in the memory cell at address 6E.
Step 5	C000	Halt.

Instruction in memory ready for execution: initial state



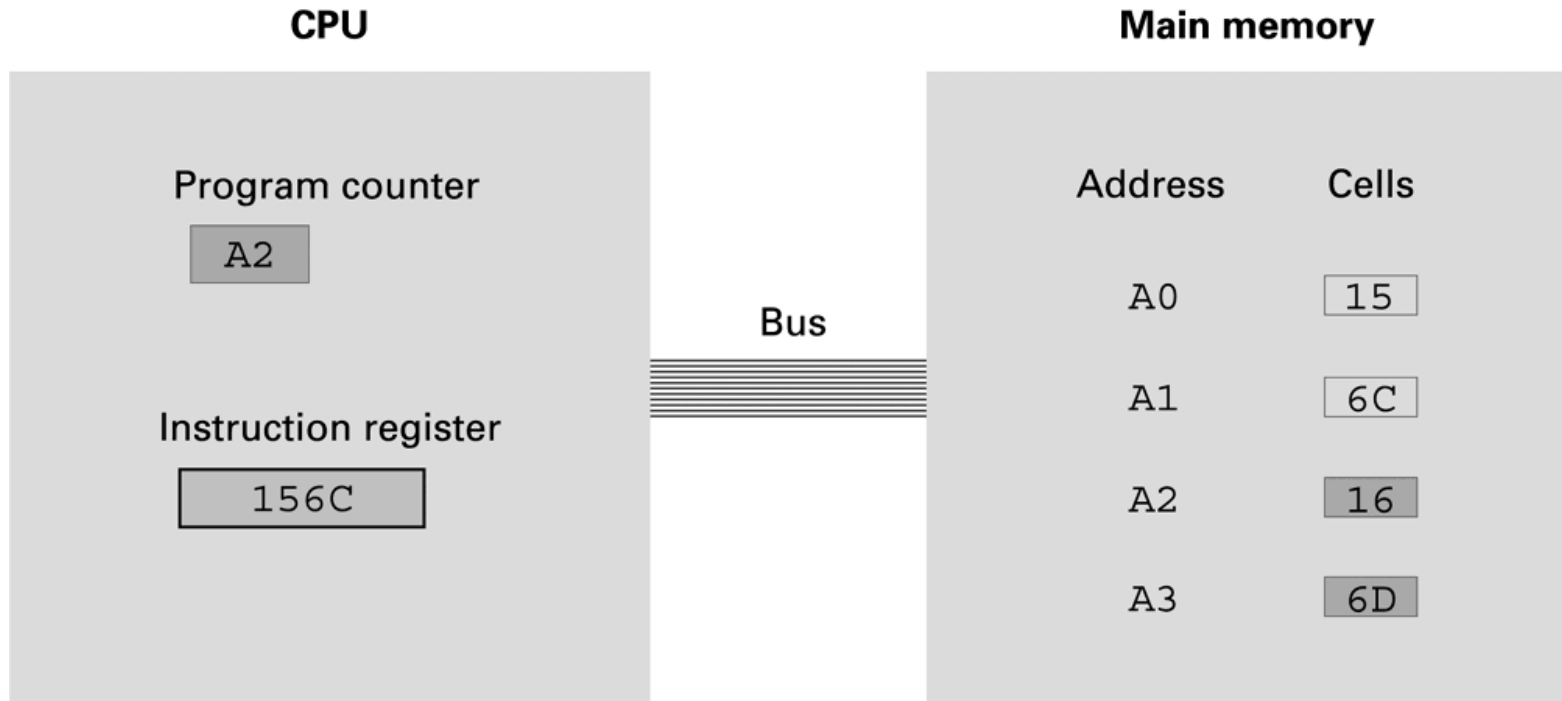
Instruction fetch: the first operation in the machine execution cycle



- a. At the beginning of the fetch step the instruction starting at address A0 is retrieved from memory and placed in the instruction register.

At the end of the instruction fetch operation, the first instruction is placed in the IR.

Updating PC for the next instruction fetch



b. Then the program counter is incremented so that it points to the next instruction.

After the instruction is placed in the IR, the CPU updates the PC.

Note: Instructions have fixed length of 16 bits (2 bytes) in the example architecture so the CPU adds 2 to PC, $A0+2=A2$.

Decoding and execution

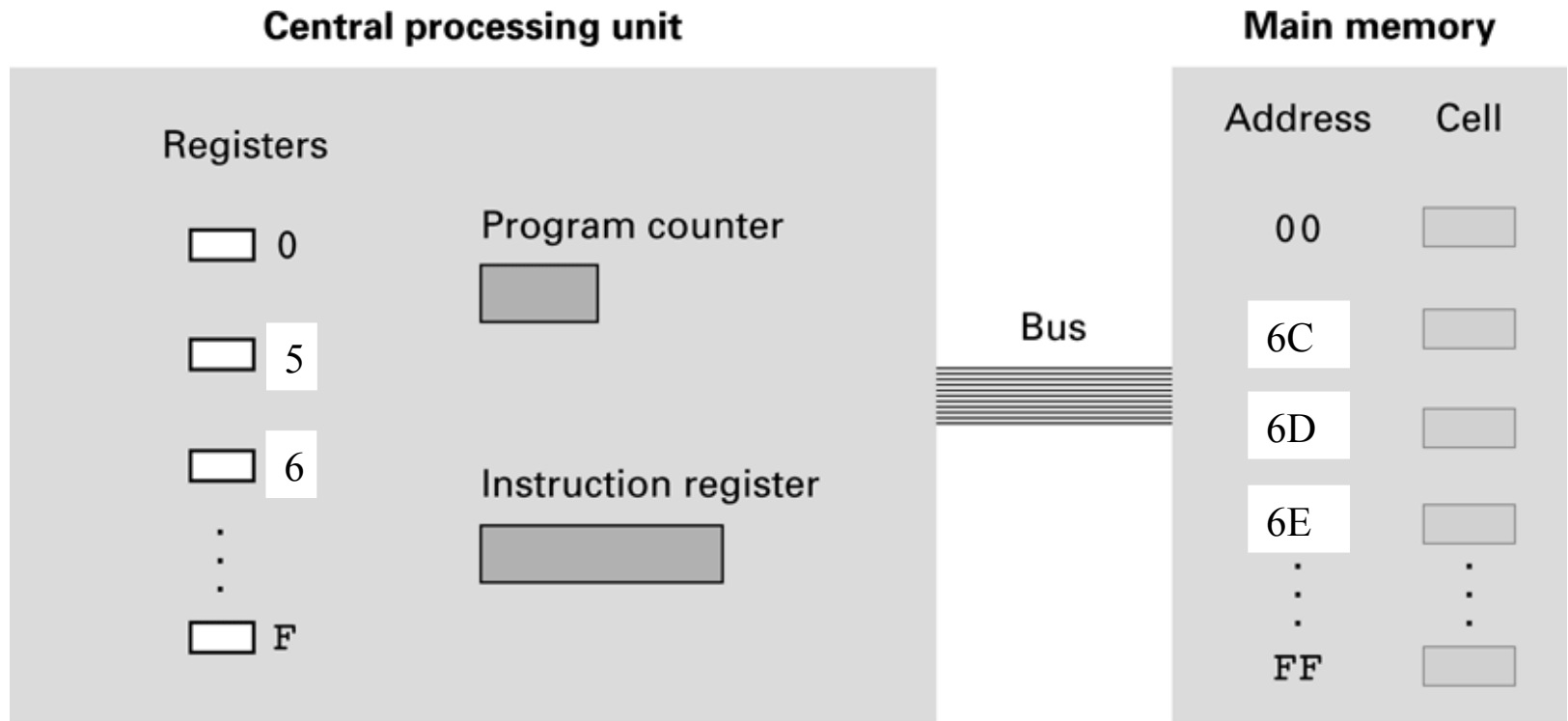
- After the first instruction is placed in IR, the decoding process starts.
 - The CPU checks the first 4 bits (opcode) of the instruction and concludes that it is to load register 5 (R5) with the content in memory address 0x6C.
- Execution cycle: the CPU loads the content in memory address 0x6C, and place it in R5.
- So far, the first execution cycle is completed and the CPU goes to the next fetch cycle.

Operation continues: what happened?

- After the first two instructions are executed
 - Data in memory address 0x6C is loaded into register R5.
 - Data in memory address 0x6D is loaded into register R6.
 - PC is updated to fetch the instruction “5056”, which is an arithmetic/logic instruction to add two values in registers.

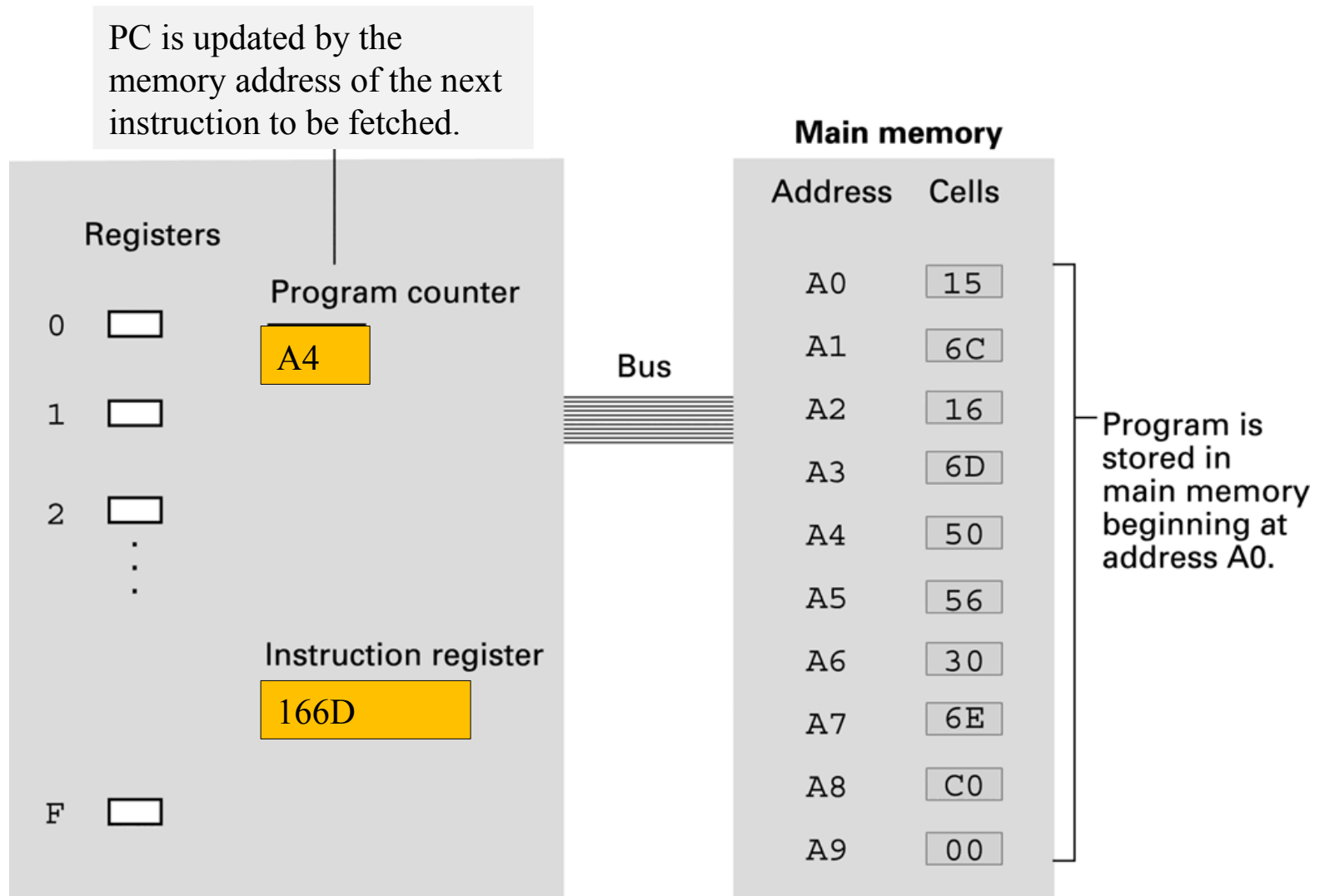
Let's continue the operations.

Data in memory for loading

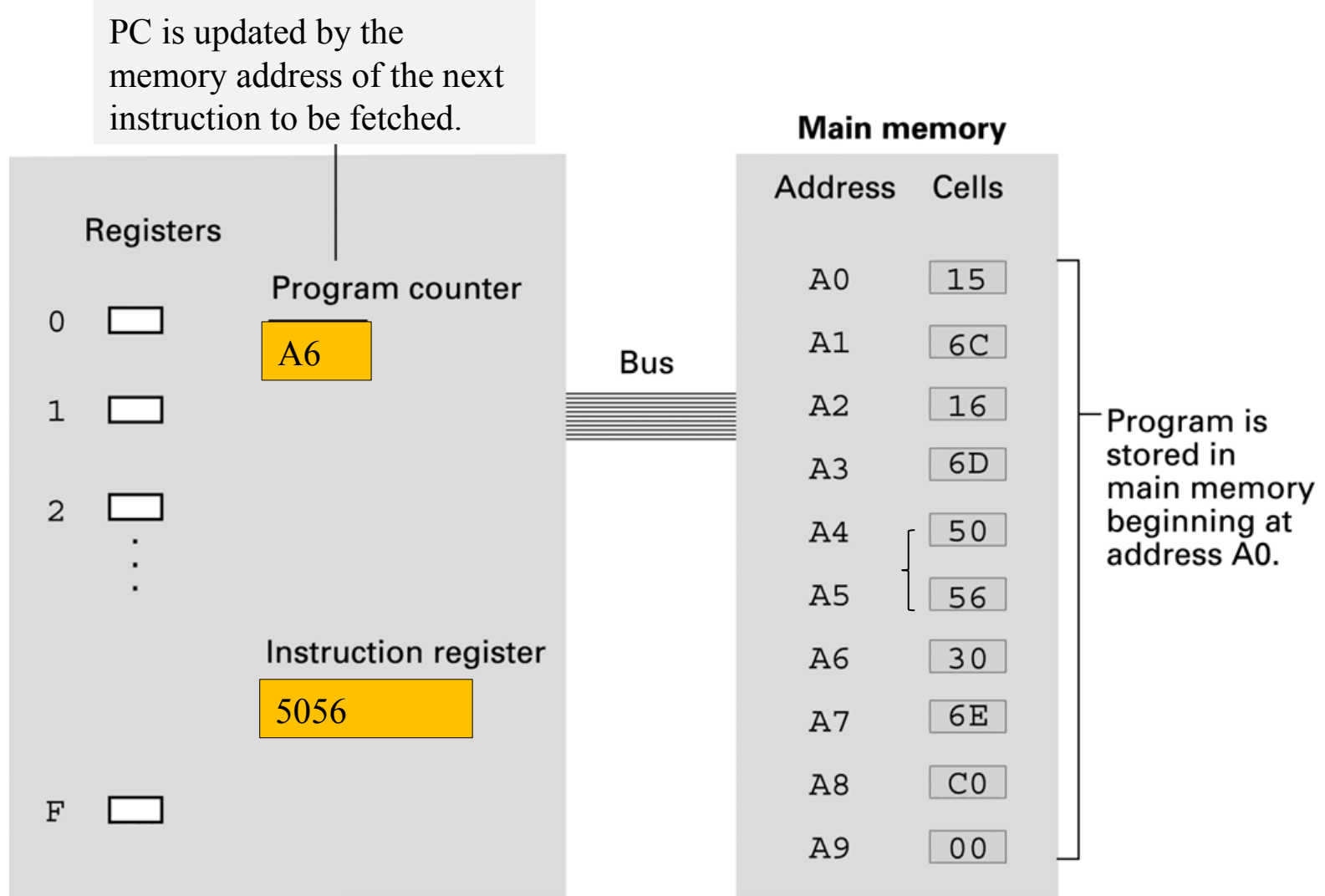


- After the first two load instructions are executed, R5=0x20, and R6=0x43
- What is in the PC?

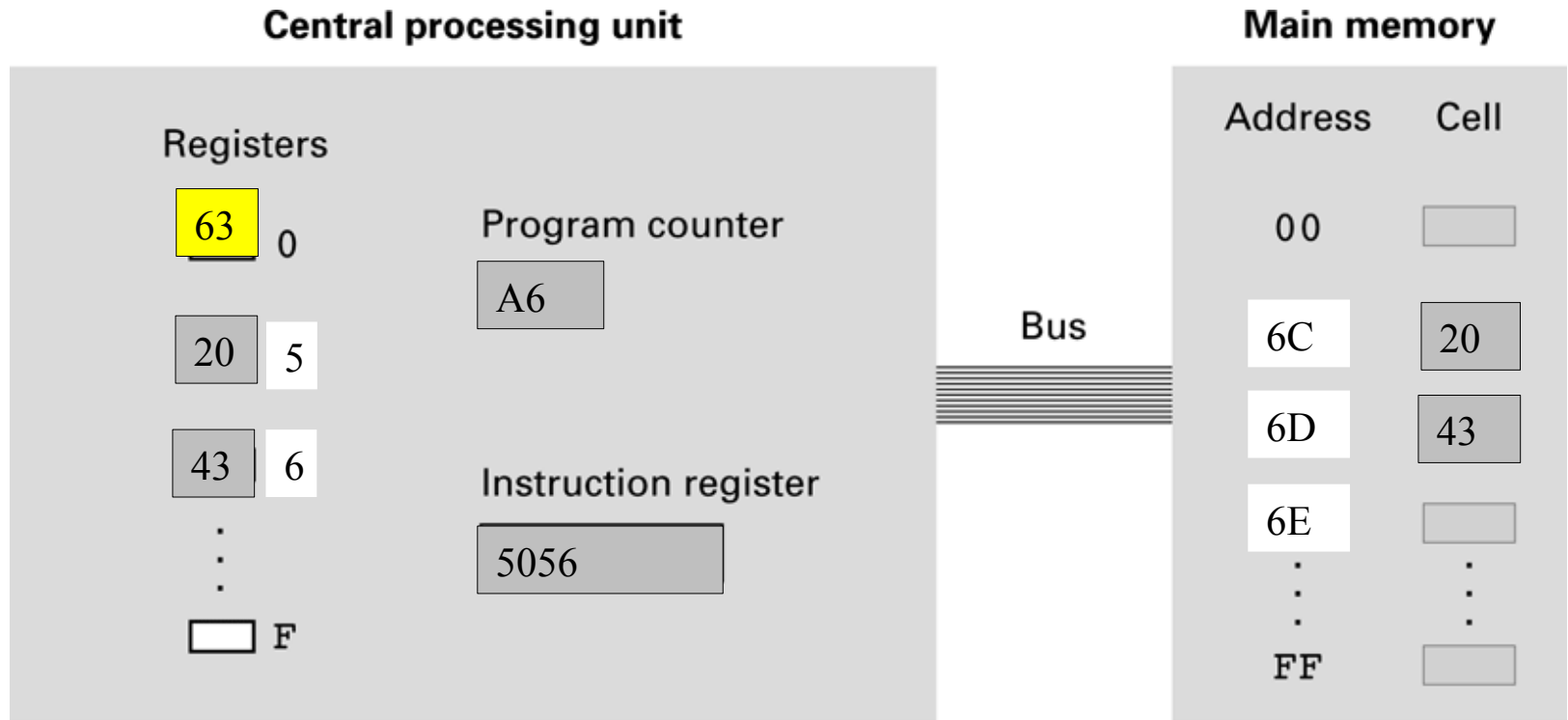
PC is updated when the second instruction is loaded



When the ADD instruction is loaded

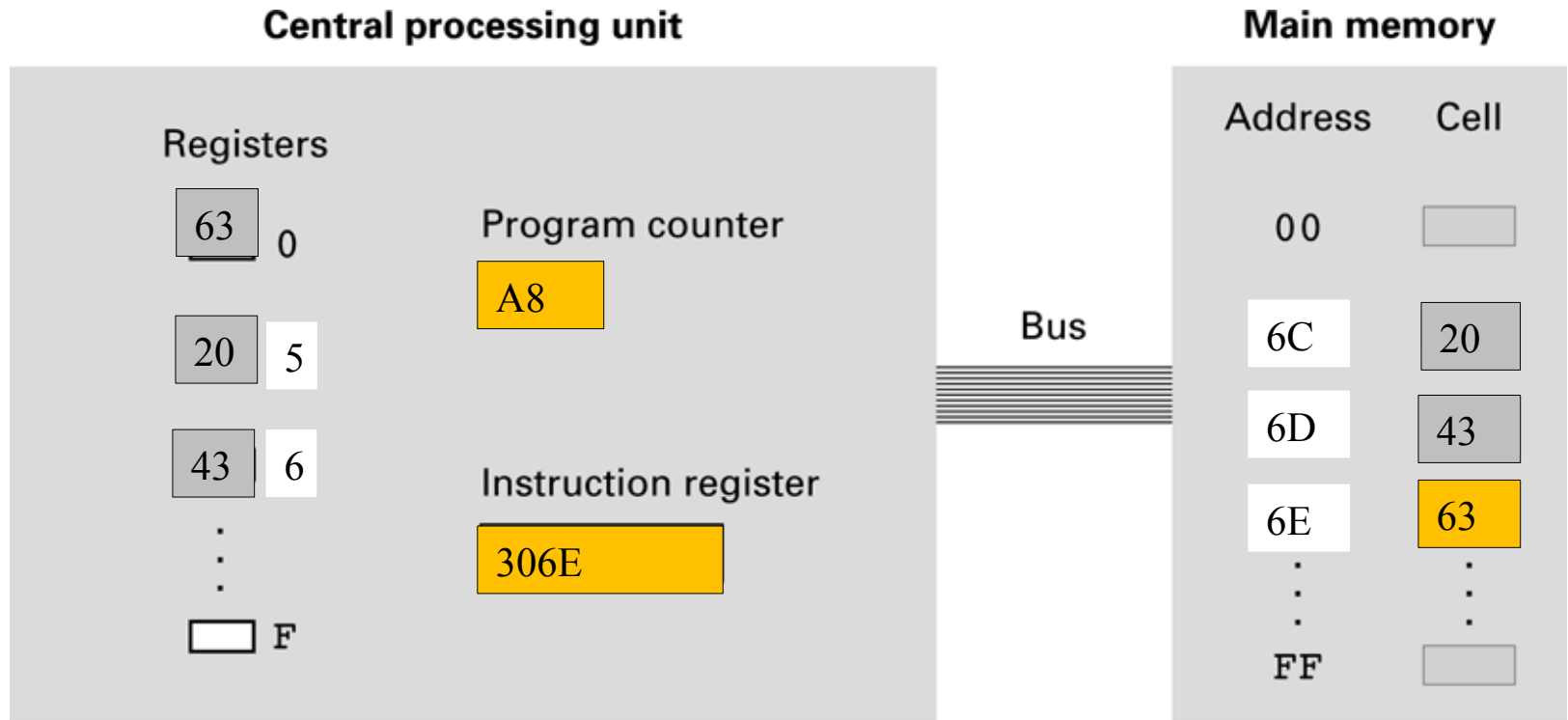


When the ADD instruction is executed



- The contents in R5 and R6 are added, and the result is placed in R0.

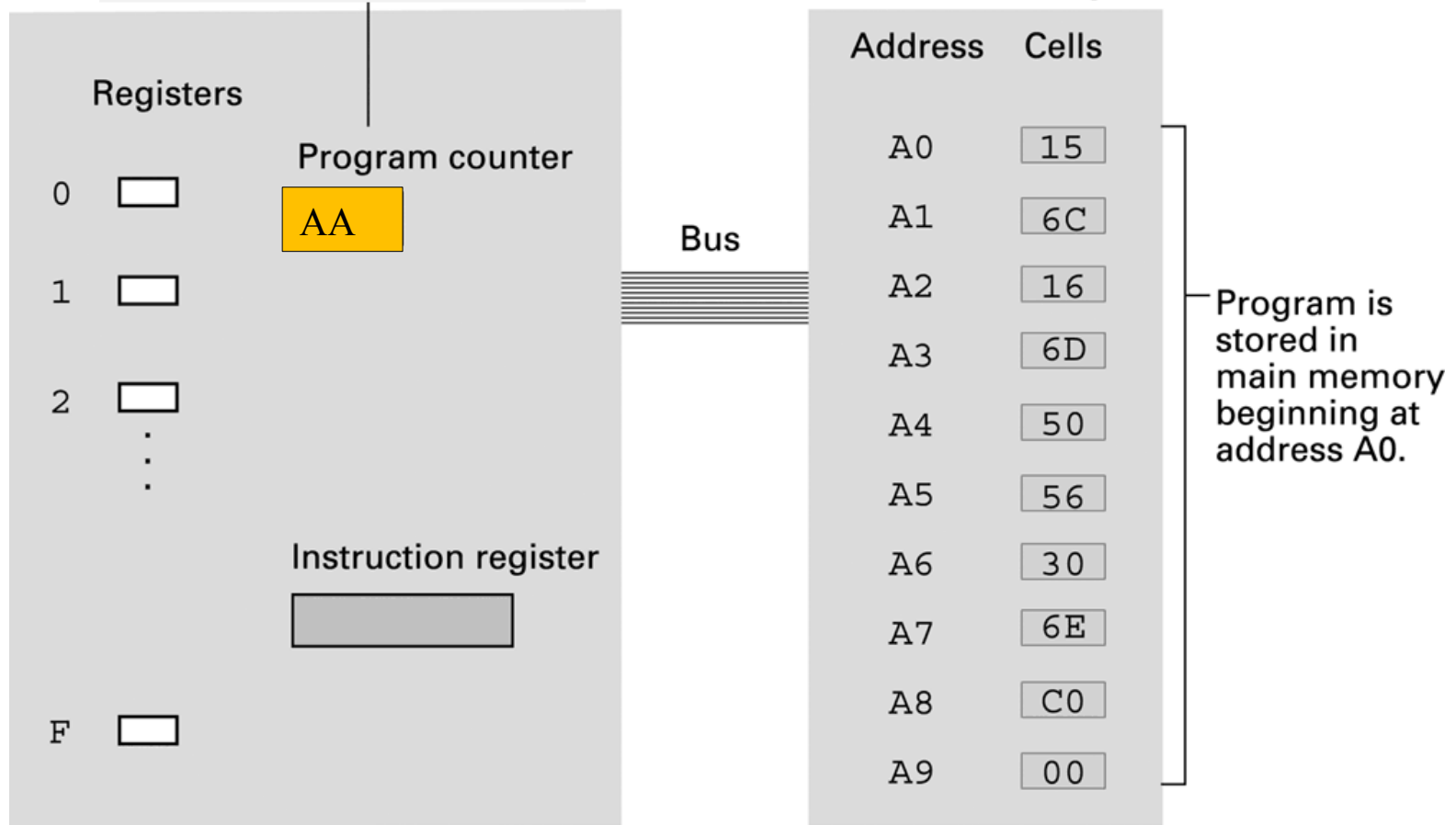
When the STORE instruction is executed



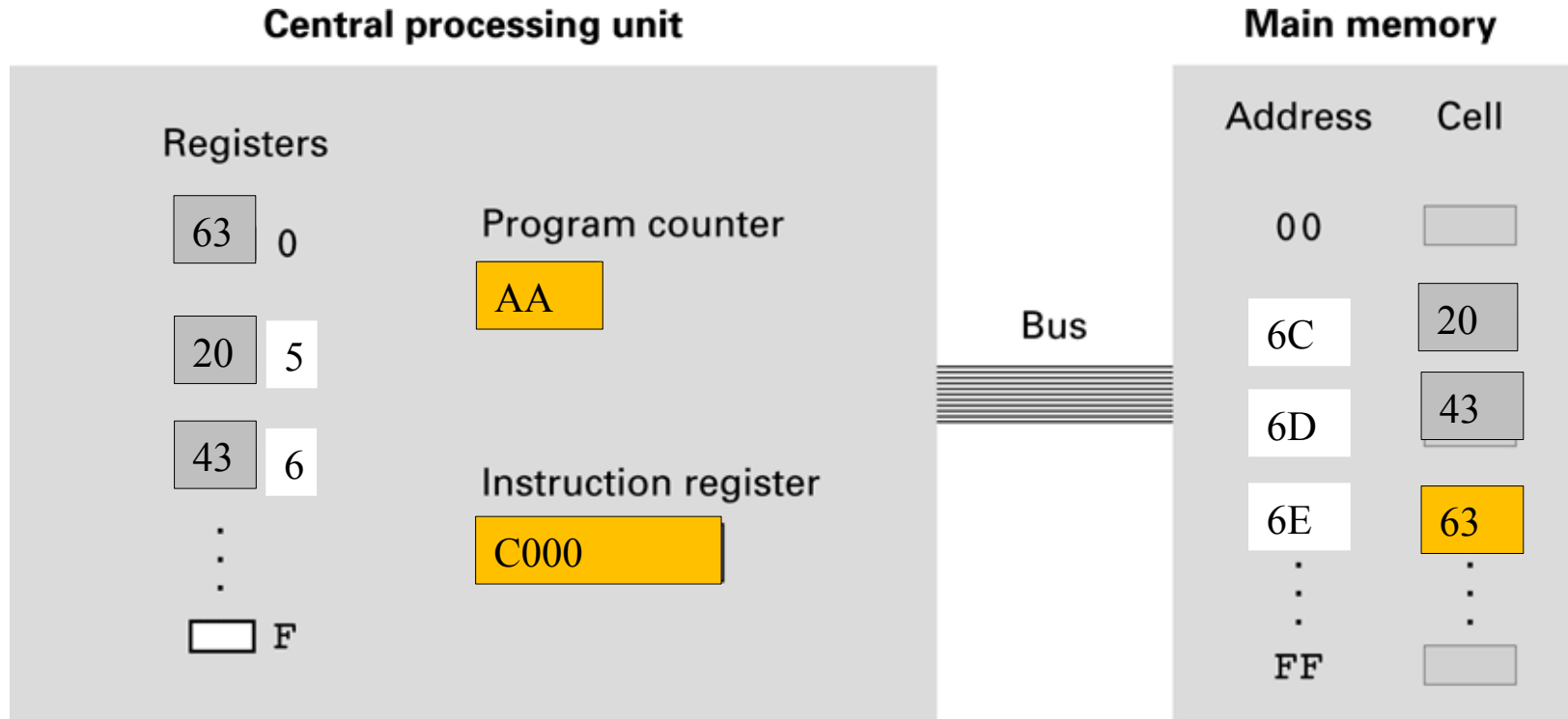
- The content in R0 is stored back to the main memory of address 6E.

What is in PC when the machine halts?

PC is updated by the memory address plus 2, which is $A8+02=AA$.



The final state of the machine



- The content in R0 is stored back to the main memory of address 6E.

End of the topic

- Please summarise what you have learned under the topic of Data Manipulation (refer to the lecture notes of weeks 3&4(1) and Chapter 2 in the book of Computer Science – An Overview).
- Make sure you complete the exercises (formative assessment) in a separate file, named as “Self-assessed coursework – data manipulations” in the week 3 folder (it is also placed in the week 4 folder) on the Blackboard.

End of the topic – online emulator

- A visualisation emulator, Virgule, is available online at <https://eseo-tech.github.io/emulsiV/>
- It implements a subset of RISC-V instructions. There are some examples in the simulator. You may play with them to see how data is manipulated in CPU-memory during instruction execution.
- You can find the instruction set of Virgule at <https://eseo-tech.github.io/emulsiV/doc/>